Fog and Cloud Computing

Better Buses

Team members
Andrea Jesus Soranzo Mendez (267339)
Andrea Precoma (267071)

University of Trento - A.Y. 2025/2026

Table of content

Images

1. Abstract

A comprehensive network of IoT sensors deployed across the urban bus fleet in Trento continuously captures real-time traffic and transit data on an hourly basis. By embedding these sensors directly within every public bus and bus stop, the system precisely calculates delays and gathers critical metrics, including peak traffic periods and recurring congestion hotspots throughout the city. This rich dataset serves multiple strategic purposes to enhance urban mobility:

- **Dynamic Route Optimization** - algorithms can analyze congestion patterns to identify alternative, more efficient paths, ensuring buses bypass traffic and arrive on time.
- **Infrastructure Strategic Planning** - engineers can identify high-friction zones where the existing road network fails to support the traffic volume. This allows for the targeted construction of new infrastructure, such as roundabouts, overpasses, or dedicated bus lanes (specifically designed to eliminate systemic congestion).
- **Data-Driven Schedule Adjustments** - transit authorities can restructure and optimize the static timetables posted at bus stops, aligning them with actual traffic realities.
- **Enhanced Passenger Experience** - the data can power a dedicated mobile application for students and commuters, providing real-time bus tracking, accurate ETA predictions, and seamless commute planning from anywhere, at any time.

2. Architecture

To realize this project, we selected a three-tier Fog Computing architecture. This hybrid approach balances local processing with heavy cloud analytics to ensure low latency, efficient bandwidth usage, and high scalability. The main task of the sensors is just to collect all the relevant data and then send them to a local and centralized server that does all the computing and calculations. After that, the data is sent to the cloud, which comprehends all the data and provides a monitoring interface with graphs and predictions in order to make all information humanly readable.

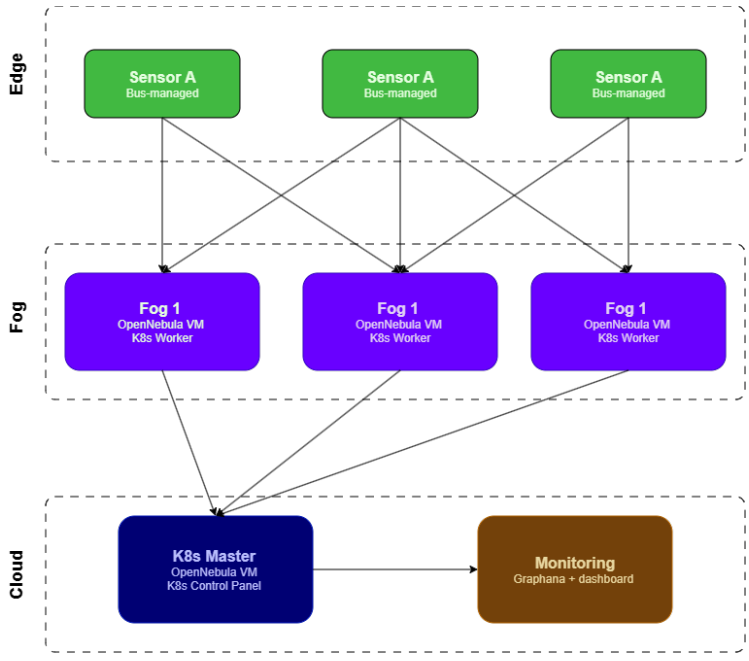

Figure 1: System architecture

2.1. Layers

2.1.1. Edge Layer

The primary role of the bus-mounted sensors is localized data ingestion. They act as lightweight edge devices responsible for continuously collecting raw transit data (such as GPS coordinates, time stamps, and speed variations) without overloading local processing capacity. Instead those mounted in the stops serves like a checkpoint station in order to track automatically at which stop and what what time a bus arrived.

2.2. Fog Layer

Once collected, these raw data packages are transmitted to localized fog nodes and regional servers. This layer performs the initial data filtering, aggregation, and critical real-time calculations, such as computing immediate route delays. By handling processing at the fog level, we minimize data transmission costs and enable rapid localized response times.

2.3. Cloud Layer

Finally, the pre-processed data is forwarded to a centralized cloud platform. The cloud orchestrates large-scale data aggregation, historical analysis, and can run predictive machine learning models. Crucially, the cloud layer hosts a comprehensive monitoring interface, converting complex datasets into human-readable dashboards, real-time graphs, and predictive analytics for stakeholders and urban planners.

2.4. Technology Stack

Speaking of the implementation we decided to mainly stick with those discovered and studied during the lectures.

2.4.1. OpenNebula

OpenNebula serves as the core cloud hypervisor and cloud management platform, orchestrating the creation, deployment, and management of our virtualized infrastructure. Given the scale of the physical deployment, OpenNebula allows us to build a high-fidelity simulation environment by provisioning distinct Virtual Machine to emulate each physical component of our architecture. This virtualized ecosystem is divided into two primary node configurations:

- **Fog Layer Emulation** - dedicated VMs are configured to simulate the local fog nodes like specified in Section 2.1.
- **Application & Visualization Layer Emulation** - a separate cluster of VMs is provisioned to host the centralized cloud services. This includes the control plane for managing the fog nodes, database systems, analytics engines, and the web servers responsible for rendering the public-facing monitoring interfaces, real-time dashboards, and data visualizations.

2.4.2. Kubernetes

To manage our data-processing applications within the fog layer, we deploy a Kubernetes cluster directly on top of the virtualized infrastructure provisioned by OpenNebula. This container orchestration strategy separates the infrastructure into a centralized Control Plane and a resilient Worker Layer. A single, dedicated OpenNebula virtual machine is allocated to act exclusively as the Kubernetes Control Plane (Master Node). This node serves as the brain of the cluster, executing core components such as scheduler, and controller manager. The other VMs are configured as Kubernetes Worker Nodes. These nodes will host the application workloads encapsulated within Pods. These worker pods execute the microservices responsible for real-time data processing and

filtering. By leveraging Kubernetes on top of our fog nodes, the system inherits robust self-healing mechanisms to ensure uninterrupted data streams and guarantees Pod replication and Workload rescheduling.

2.4.3. Grafana

To bridge the gap between complex data streams and actionable insights, we deploy Grafana on a dedicated virtual machine instance within our centralized cloud layer. This component acts as our universal data visualization and observability hub, designed to aggregate disparate metrics and present them through an intuitive, accessible, and human-readable web interface.

2.4.4. Security

For the security aspect we decided to adopt two main level of security.

2.4.4.1. Container Hardening

To minimize the attack surface of our data-processing microservices, we apply strict Container Hardening techniques. This ensures that each container is as resilient as possible against exploitation:

- **Privilege Escalation Prevention** - containers are configured to run as non-root users, stripping away unnecessary administrative capabilities.
- **Minimal Image Footprint** - we utilize lightweight base images to remove unnecessary binaries, libraries, and functionalities that could be leveraged by an attacker.
- **Resource Constraints** - defining CPU and memory limits to prevent "noisy neighbor" effects or Denial of Service (DoS) attacks originating from a compromised container.

2.4.4.2. Kubernetes Network Policies

By implementing Network Policies, we enforce a "Zero Trust"-like posture between services:

- **Namespace Isolation** - distinct environments are isolated into dedicated namespaces.
- **Granular Traffic Control** - we define explicit "allow-lists" for Pod-to-Pod communication. This ensures that a compromised sensor-data pod cannot laterally communicate with the visualization database or the control plane unless explicitly authorized.

2.4.4.3. OpenNebula Security Groups and Virtual Firewalls

The final layer of defense is managed at the hypervisor level through OpenNebula Network Policies. This acts as a perimeter firewall for the virtualized infrastructure:

- **Ingress/Egress Filtering** - we define security groups that strictly control which IP addresses and ports are accessible from outside the virtual network (e.g. allowing only HTTPS traffic to the Grafana VM).
- **Node-Level Isolation** - by restricting communication at the VM level, we ensure that even if the Kubernetes networking is bypassed, the underlying virtual nodes remain protected from unauthorized external probes or inter-node interference.